

Arqui - Setup macOS docker

[Configuración del container](#)

[Instalar Docker](#)

[Crear el container](#)

[Instalar dependencias](#)

[Compilación en el container](#)

[Script de build](#)

[Ejecutar en el container](#)

[Debugging](#)

[Extras](#)

[Generación de Makefiles](#)

La forma más sencilla de trabajar con programas para arquitectura x86 (Intel) en la materia es usar una VM, pero es una manera incómoda de trabajar, usa muchos recursos, puede ser muy lenta y nos impide usar el entorno de desarrollo (editor, terminal) que ya tenemos configurado.

Una forma más cómoda de hacerlo es configurar un container de Docker para compilar y ejecutar los programas. Esta guía explica cómo configurar Docker y automatizar algunos pasos de build.

Configuración del container

Instalar Docker

Para usar containers primero es necesario tener instalado Docker. El modo más fácil es mediante [Docker Desktop](#).

Como una mejor alternativa para macOS recomiendo [Orbstack](#) que tiene mejor rendimiento y menos uso de recursos (van a dejar el container corriendo en background constantemente, así que esto es importante sobre todo para las macbook air con 8GB de RAM).

Se puede descargar el instalador desde la web o instalar mediante [Homebrew](#):

```
brew install orbstack
```



Si no tienen instalado Homebrew recomiendo usarlo, facilita mucho la instalación de herramientas de desarrollo en macOS.

Crear el container

Una vez instalado Docker hay que pullear y correr el container. Cualquier imagen de Linux x86_64 sirve, personalmente uso la imagen de Arch Linux que viene con muchas de las herramientas necesarias (gcc, etc) ya instaladas y un buen package manager (Pacman):

```
docker pull archlinux:base-devel
docker run -v $PWD:/root --name arch -td -e SHARED_ROOT=$PWD -p 1337:1337 --privileged
--pid=host --cap-add=SYS_PTRACE --security-opt seccomp=unconfined --security-opt apparmor=unconfined --platform linux/amd64 archlinux:base-devel
```

El comando de `run` es bastante largo, podemos separarlo para ver qué estamos haciendo:

<code>docker run</code>	Comando base, crea un container
<code>-v \$PWD:/root</code>	Linkea un directorio compartido, el directorio actual donde se ejecuta el comando queda en el container

	como <code>/root</code>
<code>--name arch</code>	Nombre del container, se puede cambiar por el que prefieran
<code>-td</code>	Genera una terminal para el container para poder interactuar y lo deja corriendo en background
<code>-e SHARED_ROOT=\$PWD</code>	Guarda el path al directorio compartido que linkeamos a <code>/root</code> en una variable de entorno para usar más tarde
<code>-p 1337:1337</code>	Conecta un puerto (1337) al host, lo vamos a usar para gdb
<code>--privileged --pid=host ...</code>	Opciones de seguridad que evitan algunos errores al conectar gdb, probablemente overkill pero con estas funciona todo seguro
<code>--platform linux/amd64</code>	Le indica a docker que el container es x86_64 (en la mac el default es arm64)
<code>archlinux:base-devel</code>	La imagen que vamos a usar para el container



Para evitar accidentalmente modificar cosas desde el container lo mejor es compartir únicamente el directorio donde están las cosas de archi, tengan cuidado al correr el comando



Si queda algo mal se puede eliminar el container con `docker rm -f arch` y volver a crearlo

Después de correr los comandos deberían ver en la UI de Orbstack el nuevo container.

Instalar dependencias

Para usar el container vamos a necesitar algunas dependencias que no vienen instaladas: `nasm` para compilar assembly, y `qemu` para correr ejecutables (en principio se pueden correr directo sobre el container que corre con Rosetta y es más rápido, pero para conectar gdb Rosetta no funciona y hay que correr sobre qemu).

Primero hay que entrar al shell del container:

```
docker exec -it arch /bin/bash
```

Ahí entramos en un shell de Linux, y vamos a instalar las dependencias con Pacman, el package manager de Arch.

Primero hay que actualizar los repositorios, y luego instalar los paquetes:

```
pacman -Sy
pacman -S nasm qemu-base qemu-user-static
```

Ya podemos salir del container con el comando `exit`.

Compilación en el container

Script de build

Para compilar en el container lo más fácil es usar `make` para que el comando de compilación no cambie con cada binario. Con esto podemos hacer un script para correr `make` con un Makefile en el directorio actual:

```
#!/bin/bash

# get the host path to shared root from the container env var
# escape slashes to use in a sed pattern
basedir=$(docker exec arch3 env | grep SHARED_ROOT | cut -d=' ' -f2 | sed -e 's/\//\\//g')

# get working directory within container by replacing basedir with /root
# this only works if we're inside the base dir!
wdir=/root$(pwd | sed -e "s/^$basedir//")

# run make in the current working directory
docker exec -it -w $wdir arch make $@
```

Para usar este script hay que estar dentro del directorio compartido, pero permite usar `make` en el container desde cualquier subdirectorio del mismo. Todos los parámetros del script se pasan a `make`, con lo cual podemos hacer por ejemplo `docker_make.sh clean` directamente.

 Recomiendo agregar este script al `PATH` para poder usarlo de cualquier lado. También se puede usar un shell alternativo como `fish` que facilita crear funciones o scripts globales.

Ejecutar en el container

Para ejecutar el binario compilado en el container de docker usamos de nuevo `docker exec`:

```
#!/bin/bash

# get the host path to shared root from the container env var
# escape slashes to use in a sed pattern
basedir=$(docker exec arch3 env | grep SHARED_ROOT | cut -d=' ' -f2 | sed -e 's/\//\\//g')

# get working directory within container by replacing basedir with /root
# this only works if we're inside the base dir!
wdir=/root$(pwd | sed -e "s/^$basedir//")

# run executable in the current working directory
echo "Running $1..."
docker exec -it -w $wdir arch $@
```

Este script toma como parámetros el nombre del ejecutable y cualquier parámetro que tome, como si lo estuviéramos llamando de la terminal.

Debugging

Para poder conectar gdb al binario en docker por un problema de Rosetta no podemos hacerlo directamente, hay que correrlo en qemu. Hacemos otro script para ejecutar con debug:

```
#!/bin/bash

# get the host path to shared root from the container env var
```

```
# escape slashes to use in a sed pattern
basedir=$(docker exec arch3 env | grep SHARED_ROOT | cut -d=' ' -f2 | sed -e 's/\//\\//g')

# get working directory within container by replacing basedir with /root
# this only works if we're inside the base dir!
wdir=/root$(pwd | sed -e "s/^$basedir//")

# qemu executable, change to qemu-x86_64-static for 64 bit exe
qemu_exec=qemu-i386-static

# debug port
port=1337

# get host side ip address for container
container_ip=$(docker inspect -f '{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}' arch)

echo "Running $1 with debug server on port $port"
echo "Run gdb with command 'target remote $container_ip:$port' to debug"

# nuclear option (ensure no running qemu instance)
docker exec arch killall -9 $qemu_exec 2> /dev/null

# run gdbserver on container
docker exec -ti -w $wdir arch $qemu_exec -g $port $@
```

Luego la ejecución pausa en la primera instrucción y podemos conectar gdb con el comando que imprime en la terminal.

Extras

Dejo un script completo que hace todo junto con opciones de línea de comando. El script es fish, traducirlo a bash queda como ejercicio para el lector (alternativa: usar fish, es un buen shell 🐟)

```
function docker_make
    set -l container_name $argv[1]
    set -l target $argv[2]

    set -l basedir (docker exec arch3 env | grep SHARED_ROOT | cut -d=' ' -f2 | sed -e 's/\//\\//g')
    set -l wdir /root(pwd | sed -e "s/^$basedir//")

    docker exec -w $wdir $container_name make $target
end
```

```
function make_run_x86
    argparse -i 'g/debug' 'c/container-name=' 'v/verbose' 'q/qemu' 'x/x86_64' -- $argv
    or return

    set -l container_name $_flag_c
    set -l target $argv[1]
    set -l args $argv[2..]
```

```

    set -l basedir (docker exec arch3 env | grep SHARED_ROOT | cut -d'=' -f2 | sed -e
's/\//\//g')
    set -l wdir /root(pwd | sed -e "s/^\$basedir//")

if set -ql _flag_v
    docker_make $container_name $target
else
    docker_make $container_name $target > /dev/null
end

set -l qemu_exec "qemu-i386-static"
if set -ql _flag_x
    set qemu_exec "qemu-x86_64-static"
end

if set -ql _flag_g
    set -l port 1337
    if set -ql _flag_p
        set port _flag_p[1]
    end

    set -l container_ip (docker inspect -f '{{range.NetworkSettings.Networks}}{{.IP
Address}}{{end}}' $container_name)

    echo "Running $argv[1..] with debug server on port $port"
    echo "Run gdb with command 'target remote $container_ip:$port' to debug"

    # Nuclear option
    docker exec $container_name killall -9 $qemu_exec 2> /dev/null

    # Run gdbserver on container
    docker exec -ti -w $wdir $container_name $qemu_exec -g $port $target $args
else if set -ql _flag_q
    echo "Running $argv[1..] [QEMU]..."

    # Nuclear option
    docker exec $container_name killall -9 $qemu_exec 2> /dev/null

    # Run qemu on container
    docker exec -ti -w $wdir $container_name $qemu_exec $target $args
else
    echo "Running $argv[1..]..."

    docker exec -ti -w $wdir $container_name ./ $target $args
end
end

```

Generación de Makefiles

Como escribir todos los Makefile a mano puede ser engorroso, dejo otro script de fish que genera templates para cada ejercicio y los suma a un Makefile existente:

```

#!/opt/homebrew/bin/fish

# Usage: ./mk.fish [-cCsS] [-a 32/64] 01 hello [libs]
argparse -i 'c/make-c' 'C/main-c' 's/make-asm' 'S/main-asm' 'a/arch=' -- $argv

set -l arch "32"
if set -ql _flag_a
    set arch $_flag_a[1]
end

set -l num $argv[1]
set -l name $argv[2]
set -l dir_name (string join _ $num $name)

set -l libs $argv[3..]

mkdir $dir_name

set -l exec_deps "$dir_name/main.o"

# Create ASM source file
if set -ql _flag_S
echo "section .text
global main

main:

    mov eax, 0        ; exit with code 0, success
    ret

section .data" >> $dir_name/main.s
else if set -ql _flag_s
echo "section .text

section .data" >> $dir_name/$name.s
set exec_deps (string join " " $exec_deps "$dir_name/$name.o")
end

# Create C source file
if set -ql _flag_C
echo "#include <stdio.h>

int main(int argc, const char** argv) {
    printf("\nHello, world!\n\n");

    return 0;
}" >> $dir_name/main.c
else if set -ql _flag_c
echo "" >> $dir_name/$name.c
set exec_deps (string join " " $exec_deps "$dir_name/$name.o")
end

```

```

# Update makefile
# Main executable target
echo "
$name: $exec_deps $libs
    gcc -g -m$sarch $exec_deps $libs -o $name" >> Makefile

# Main obj target
if set -ql _flag_S
echo "
$dir_name/main.o: $dir_name/main.s
    nasm -g -F dwarf -f elf$sarch $dir_name/main.s -o $dir_name/main.o" >> Makefile
else if set -ql _flag_C
echo "
$dir_name/main.o: $dir_name/main.c
    gcc -c -g -F dwarf -m$sarch $dir_name/main.c -o $dir_name/main.o" >> Makefile
end

# Auxiliary source target
if set -ql _flag_s
echo "
$dir_name/$name.o: $dir_name/$name.s
    nasm -g -F dwarf -f elf$sarch $dir_name/$name.s -o $dir_name/$name.o" >> Makefile
else if set -ql _flag_c
echo "
$dir_name/$name.o: $dir_name/$name.c
    gcc -c -g -F dwarf -m$sarch $dir_name/$name.c -o $dir_name/$name.o" >> Makefile
end

```

El script genera dentro de un mismo makefile un nuevo target para cada ejercicio. Luego para compilarlo `make_run_x86` `<nombre> [<args...>]`.